CHAPTER 1

DESIGNING AN ONLINE BLACKJACK GAME

1 Getting Ready: Online Blackjack Game 2
2 Requirements for the Online Blackjack Game 7
3 Use Case Diagram for the Online Blackjack Game . . . 9
4 Class Diagram for the Online Blackjack Game 12
5 Activity Diagram for the Online Blackjack Game 23
6 Code for the Online Blackjack Game 24
7 Summary: Online Blackjack Game 31
8 Key Insights and Tips: Designing an Online Blackjack Game 35

This chapter explores how to design an online blackjack game, covering requirements, use case, class, activity diagrams, and implementation code.

1 Getting Ready: Online Blackjack Game

1.1 Problem definition

Blackjack is a popular casino card game where multiple players compete individually against a dealer. The main objective is to have a hand value closer to 21 than the dealer without exceeding it ("busting"). The game uses one or more standard decks of cards. Each card has a specific value: number cards (2-10) are worth their face value, face cards (Jack, Queen, King) are worth 10 points, and the Ace can count as either 1 or 11, whichever benefits the hand most. The best possible hand, a "Blackjack," consists of an Ace and a 10-point card (10, Jack, Queen, or King).
At the beginning of each round, players place bets, and players and the dealer deals two cards to each. Players ' cards are face up, while the dealer has one face up and one face down card. Players can choose to "hit" (draw an additional card) or "stand" (end their turn) in an attempt to get closer to 21 without busting. If a player's total exceeds 21, they lose immediately. After all players have finished, the dealer reveals their hidden card and draws according to predefined rules. Winners are determined by comparing hand values: the player wins if their total is higher than the dealer's without busting.
In this LLD interview case study, you'll focus on:

- Modeling the core game mechanics of Blackjack, including card values, player and dealer actions (hit, stand), betting, and win/loss conditions.
- Designing an interactive flow for multiple players playing concurrently against the dealer, properly handling card dealing, turns, and bet outcomes.
- Ensuring the system can reliably manage hands, bets, and game state, while enforcing all Blackjack rules (such as Blackjacks, busts, ties, and dealer drawing rules).
- Building a foundation that can be extended for features like multiple tables, different deck sizes, or online multiplayer support.

  Note: This system model is adaptable for online casinos, learning simulators, or any platform looking to provide a scalable Blackjack game experience.

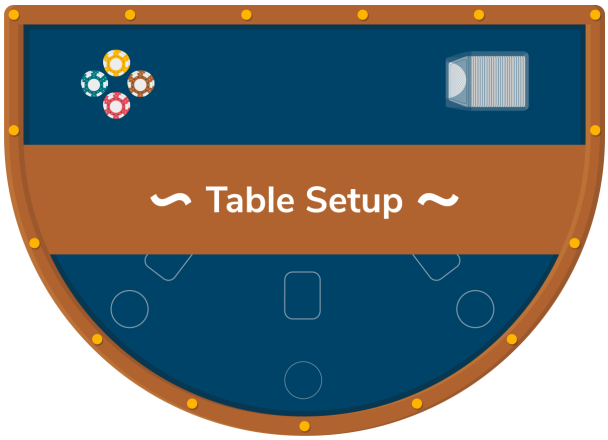
The following slide represents how to play Blackjack:

1.2 Expectations from the interviewee

There are several components in a Blackjack game, each with specific constraints and requirements. The following provides an overview of some of the main expectations that the interviewer will want to hear you discuss in more detail during the interview.

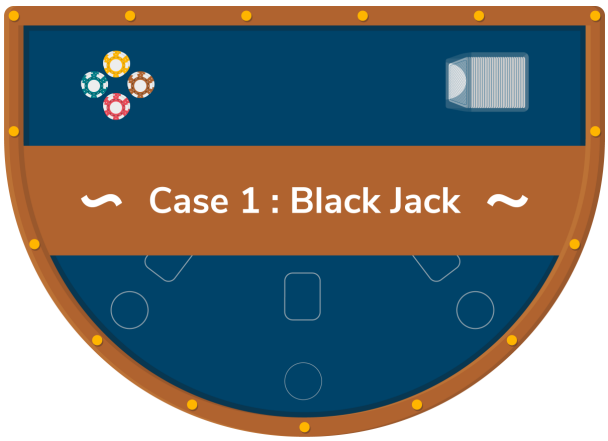
Players in the game

To get a better understanding of the game and the players of the game, you may ask the interviewer the following set of questions:

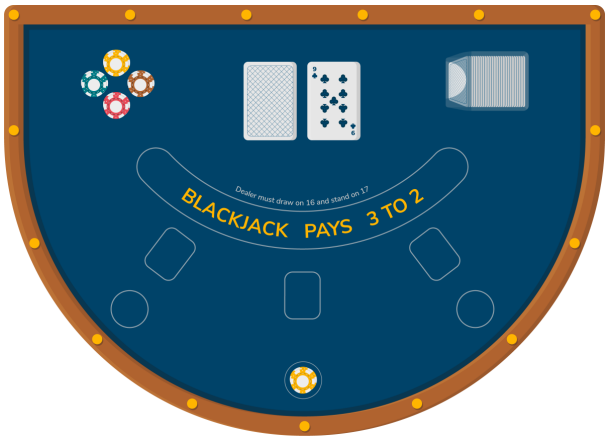


(a) Blackjack table setup

(b) The dealer's end and player's end



(a) Blackjack case represents if the player's first two cards' sum is equal to 21



(b) The dealer deals two cards to themselves, one card facing up and the other card facing down

(a) The dealer will deal two cards to player (both cards facing up)

(b) Player wins the game because their first two cards' sum is equal to 21 and gets 150 percent profit

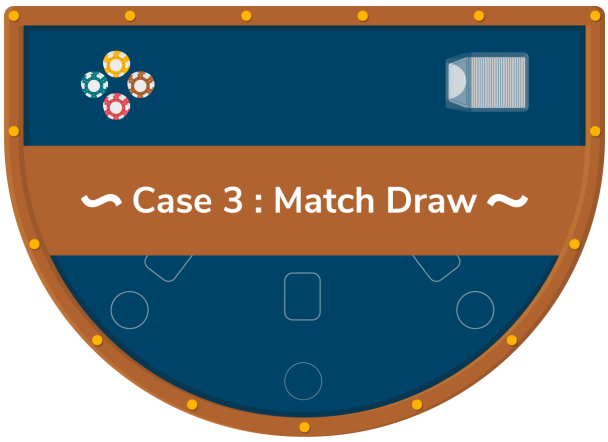
(a) Dealer wins if the dealer card sum is greater than player's card sum

(b) The dealer deals two cards to themselves and two cards to the player

(a) The player hits and draws an additional card (8)

(b) After the player stands, the dealer exposes their second card

(a) Player loses the game and the bet because player cards' sum (18) is less than dealer cards' sum (19)



(b) The game results in a draw if the player card sum is equal to the dealer card sum

(a) The dealer deals two cards to themselves and two cards to the player

(b) The player hits and draws an additional card (8)

(a) After the player stands, the dealer exposes their second card

(b) The game results in a draw because player cards' sum (18) is equal to the dealer cards' sum (18)

(a) The player wins if their card sum is greater than the dealer card sum

(b) The dealer deals two cards to themselves and two cards to the player

(a) The player hits and draws an additional card (8)

(b) The player wins the game and the bet(100%) because player cards sum is greater than the dealer cards sum

1. How many players can play Blackjack?
2. Can players play the game against each other?

Point dynamics

The rules related to the number of points also need to be clarified by the interviewer; therefore, you may ask the questions listed below:

1. How many points can the player or the dealer hit the card?
2. What will happen if the dealer and the player get the same points?

Card limit

The interviewer would expect you to ask about the maximum number of cards the player can have. Therefore, you may ask a question listed below:

1. Is there a limit on the number of cards the player takes?

1.3 Design approach

We will design this online Blackjack game using a bottom-up design approach. To achieve this, we will follow these steps:

- First, we'll identify and design the smallest, core entities such as `Card`, `Hand`, and `Player` (including the dealer as a special player), clarifying their responsibilities and interactions.
- Next, we'll model the `Deck`, `GameTable`, and `GameController` to coordinate the game flow—dealing, betting, player actions, dealer logic, and determining results.

- We will incorporate game rules (card values, Blackjacks, busts, and win conditions) into these entities' behaviors, ensuring correctness and clarity.
- We'll ensure that the design allows for scalability, supporting multiple tables and concurrent games, and enforcing a clear separation of concerns for easy maintenance and extension.
- We'll highlight how SOLID principles and relevant design patterns can help us achieve a clean, maintainable system.

1.4 Design pattern

During an interview, discussing the design patterns that an online Blackjack game falls under is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:

Which design pattern(s) should be used to design an online Blackjack game? Please elaborate on your choice(s).

Let's explore the requirements of the online Blackjack game in the next lesson.

1.5

2 Requirements for the Online Blackjack Game

This lesson outlines the functional and operational requirements for the online Blackjack game. Identifying and understanding requirements is essential to defining the system's scope and ensuring a robust, user-friendly design.
We'll use the notational convention to identify each requirement with a unique label "Rn," where "R" is short for requirement and "n" is a natural number.

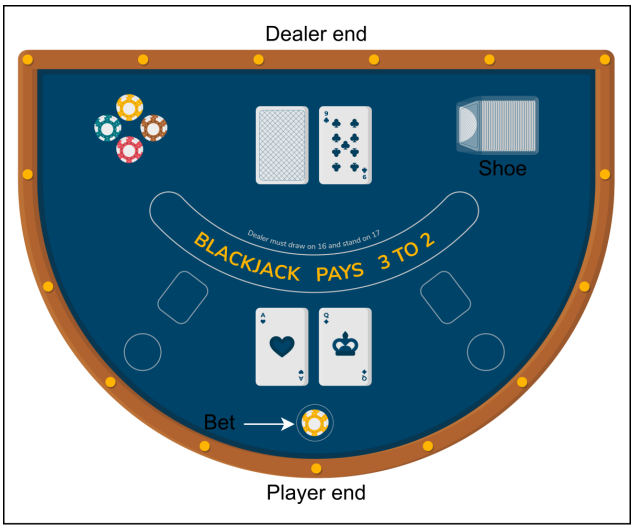
2.1 Requirement collection

The requirements for the Blackjack design problem are defined below:

- **R1:** The system must support a shoe containing one or more standard decks of cards.
- **R2:** Each deck consists of 52 cards in four suits (Hearts, Diamonds, Clubs, Spades), each containing 13 ranks: Ace, 2–10, Jack, Queen, and King.
- **R3:** Every card has a specific point value:

Card	Face Value
Ace	1 or 11
From 2 to 10	Equals the card number
Face cards (King, Queen, and Jack)	10

- **R4:** The game has two user roles: the player and the dealer.
- **R5:** Each player must place a bet at the start of every round.
- **R6:** At the start of each game round, the dealer deals two cards to each player and two to themselves.
- **R7:** Both player cards are dealt face up; the dealer's first card is face up, and the second is face down.



- **R8:** Players may hit (draw an additional card) if their hand total is less than 21 points.
- **R9:** According to standard Blackjack rules, the dealer must hit if their hand value exceeds 17 points.
- **R10:** If a player's or dealer's hand exceeds 21 points, they bust and immediately lose that round.
- **R11:** Players may stand (decline to take more cards) at any time during their turn.
- **R12:** At the end of the round, if a player's hand total is higher than the dealer's and does not exceed 21, the player wins and receives a payout equal to their bet (1:1).

- **R13:** If a player achieves a Blackjack (Ace and a 10-point card as their initial two cards), they receive a payout of 3:2 (150% of their bet).
- **R14:** If the player and dealer have the same point total at the end of a round, it is a push; the player gets their original bet back, or may be offered a replay.
- **R15:** If a player leaves or disconnects before the end of a round, the dealer automatically wins that round.

We've identified our requirements for the problem, and in the next lesson, we will define different use cases for the online Blackjack game design.

3 Use Case Diagram for the Online Blackjack Game

Let's build the use case diagram of theonline Blackjack game and understand the relationships between its main actors and system functions. First , we'll define the different elements of our system, followed by the main actors and their associated use cases.

3.1 System

Our system is an online Blackjack game. It manages player interactions, game rounds, betting, and outcomes between players and dealers.

3.2 Actors

Now, we'll define the main actors of our Blackjack game.

Primary actor

- **Player:** The main participant in the game. A player can join games, place bets, perform in-game actions (hit, stand, resign), and manage their account (create, update, reset password, cancel membership, login/logout).

Secondary actor

- **Dealer:** The dealer manages the Blackjack table, controls the game flow, deals cards, enforces rules, and completes payouts. The dealer may also manage player accounts (e.g., block members).

3.3 Use cases

This section will define the use cases for the Blackjack game. We have listed the use cases according to their interactions with a particular actor.

Note: Some use cases will occur multiple times because they are shared among different actors in the system.

Player

- **Create account:** To register a new player profile in the Blackjack system.
- **Login/Logout:** To securely log in or out of the Blackjack game.
- **Update Account:** To edit personal account details, such as username, password, and contact information, or even cancel membership.
- **View Open Games:** Browse a list of available Blackjack games waiting for players to join.
- **Join a Game:** To enter an open Blackjack table as a participant.
- **Place Bet:** To wager a specified amount for the current round.
- **Hit:** To request an additional card from the dealer during a round.
- **Stand:** To end the turn and retain current cards, awaiting game outcome.
- **Resign Game:** To leave an active game before it is finished.

Dealer

- **Create Account:** To register a new dealer profile in the system.
- **Login/Logout:** To access or exit the dealer interface with credentials.
- **Manage Account:** To oversee player accounts, including blocking a member from a game, and cancel their memberships.
- **Create New Game:** To set up a new Blackjack table or game round.
- **View Open Games:** To monitor all open games and wait for players.
- **Create Hands:** To deal two cards to each player and the dealer at the start of a round.
- **Draw Card:** To add a card to any hand during gameplay.
- **Collect or Payout:** To settle bets by collecting losses or awarding winnings at the end of each round.

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Associations

The table below shows the association relationship between actors and their use cases.

Player	**Dealer**
Create account	Create account
Login/Logout	Login/Logout
Update account	Manage account
View open games	Create a new game
Join game	View open games
Place bet	Create hands
Hit	Draw card
Stand	Collect or payout
Resigns a game	-

Include

Include relationships that capture mandatory, reusable behavior. They help avoid repetition: shared logic is modeled as a separate use case and included wherever needed.
Examples in the online Blackjack game:

- "Draw a card" is included in both:
  - **Create hands:** When a new round starts, cards must be drawn for each participant (players and dealer).
  - **Hit:** The system draws a card when a player requests an extra card.
- "Stand" includes "Collect or payout"**:**
  - After a player stands, the game outcome is evaluated, and payouts or collections are always processed as the next step.

Extend

Extend relationships represent optional or conditional behavior that occurs only under certain conditions or as a specific extension of the base use case.
Examples in the online Blackjack game:

- "Manage an account" (for the dealer) is a general use case for managing player accounts.
  - "Block member" and "Cancel membership"are specific actions that extend "Manage an account," as they are specialized cases triggered only when required.

- "Update an account" (for the player) is the general use case for personal account changes.

  - "Cancel membership" extends "Update an account," as canceling one's membership or resetting one's password are conditional flows, not always part of every account update.

3.5 Use case diagram

Here's the use case diagram of the Blackjack game:

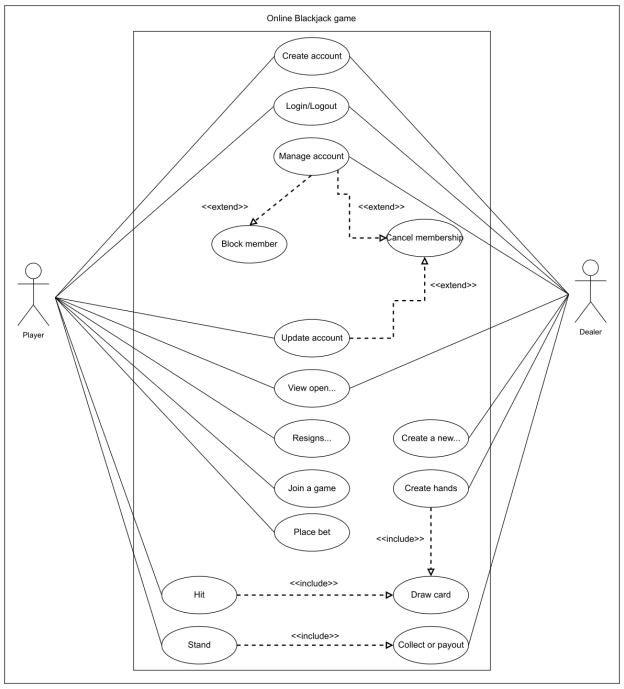


Figure 1.11: The use case diagram for the Blackjack game

In the next lesson, we will discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for the Online Blackjack Game

In this lesson, we'll create the class diagram for the Blackjack game. We will first define the core classes, then identify their relationships to fully model the game's structure according to our requirements.

4.1 Components of Blackjack

In this section, we'll define the classes for the Blackjack game. As mentioned earlier, we are following the bottom-up approach to designing a class diagram for the Blackjack game.

Card

The `Card` class encapsulates the properties of a single playing card used in Blackjack. Each `Card` object is defined by its suit (hearts, spades, clubs, or diamonds) and face value (ranging from Ace to King). The class includes methods to determine the card's Blackjack point value (with special handling for face cards and Aces) and to provide a human-readable string representation of the card.

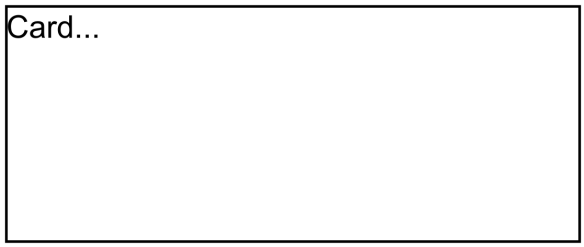


Figure 1.12: The class diagram of the Card class

R2: Each deck consists of 52 cards in four suits (Hearts, Diamonds, Clubs, Spades), each containing 13 ranks: Ace, 2–10, Jack, Queen, and King.
R3: Every card has a specific point value:

- **Ace:** values equal 1 or 11
- **Number cards from 2 to 10:** values equal to their numbers
- **Face cards:** value equals 10 for each

Deck

The `Deck` class represents a standard 52-card playing deck, consisting of all possible suit and face value combinations. Upon instantiation, a deck automatically populates with `Card` objects for each unique combination. Deck provides access to its list of cards, enabling shuffling, dealing, and integration into larger components like the `Shoe` class.

R2: Each deck consists of 52 cards in four suits (Hearts, Diamonds, Clubs, Spades), each containing 13 ranks: Ace, 2–10, Jack, Queen, and King.

Shoe

The `Shoe` class models the casino device that holds multiple shuffled decks of cards. It is the source for all card dealing during the game, ensuring a large enough card supply for extended play and fair distribution. The `Shoe` is responsible for initializing itself with the

Figure 1.13: The class diagram of the Deck class

required number of decks, shuffling all the cards together, and providing cards one at a time for dealing to players and the dealer.

Figure 1.14: The class diagram of the Shoe class

R1: The system must support a shoe containing one or more standard decks of cards.

Hand

The `Hand` class represents the collection of cards a player or dealer currently holds in a single round of Blackjack. It provides operations for adding new cards and computing the hand's total point value, with intelligent handling of the Ace's dual value (1 or 11) to optimize for the best score without busting.

Figure 1.15: The class diagram of the Hand class

R3: Every card has a specific point value:

- **Ace:** values equal 1 or 11
- **Number cards from 2 to 10:** values equal to their numbers
- **Face cards:** value equals 10 for each

R8: Players may hit (draw an additional card) if their hand total is less than 21 points.
R9: According to standard Blackjack rules, the dealer must hit if their hand value exceeds 17 points.
R10: If a player's or dealer's hand exceeds 21 points, they bust and immediately lose that round.

Player, BlackjackPlayer, and Dealer

The `Player` class is an abstract base class that encapsulates all the common attributes and behaviors shared by participants in the Blackjack game. Each player maintains identifying information, login credentials, account status, a running cash balance, and a reference to their personal details through the `Person` object. The `Player` class is also associated with a `Hand`, representing the current set of cards held by that participant in any round.

- `BlackjackPlayer`**:** This class represents a human participant who joins the Blackjack game to place wagers and compete against the dealer. Extending from the `Player` base class, `BlackjackPlayer` adds capabilities specific to casino play, such as tracking the amount of the current bet amount, updating the total cash balance after each win or loss, and managing the logic for payouts according to Blackjack's rules.
- `Dealer`**:** This class models the role of the house representative in a Blackjack game. Inheriting from the `Player` base class, `Dealer` manages the game's flow according to standard casino procedures. This includes dealing the initial cards to all participants, maintaining the dealer's hand, and executing all dealer-specific rules—such as drawing cards until reaching a minimum hand value (usually 17).

R4: Two types of users can play the Blackjack game, i.e., the dealer and the player.

BlackjackController

The `BlackjackController` class acts as the mediator between the player's actions and the core game logic. It is responsible for validating player decisions (such as whether a requested action is legal at a given point in the game) and processing those actions according to the rules.

BlackjackGameView

The `BlackjackGameView` class handles all aspects of displaying the game's state to the players. It receives updates from the game logic and presents key information such as the current hands, dealer's exposed cards, round results, and payouts. This class ensures that players are always informed of their status and the outcome of each game event in a clear and timely manner.

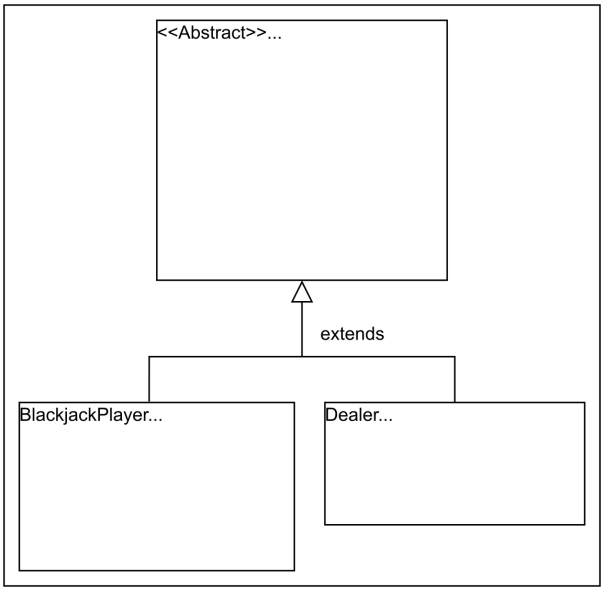


Figure 1.16: The class diagram of Player and its derived classes

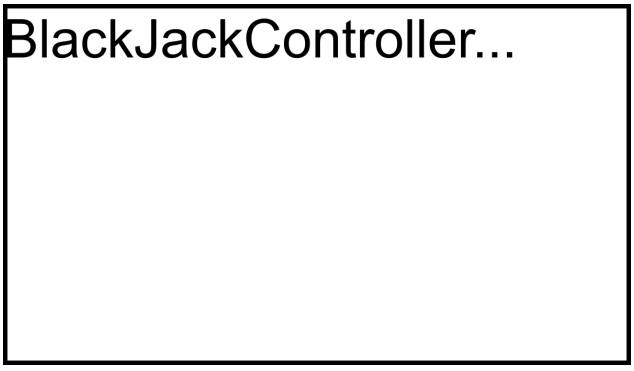


Figure 1.17: The class diagram of the BlackjackController class

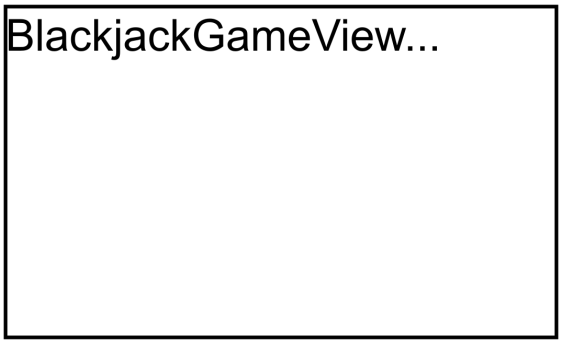


Figure 1.18: The class diagram of the BlackjackGameView class

BlackjackGame

The `BlackjackGame` class is the central orchestrator for the entire game session. It manages the main sequence of play, ensuring the correct flow of each game round. This class is responsible for initializing the game with the necessary players, dealer, and shuffled shoe of cards. It accepts wagers from players, distributes cards from the Shoe to players' and the dealer's hands, and updates the game's status throughout the round.
During gameplay, `BlackjackGame` processes all player decisions—such as hit or stand—by invoking the necessary logic and enforcing Blackjack rules. It manages the collection of lost wagers, calculates and pays out winnings, and ensures that the dealer's actions (such as revealing the hole card and drawing to 17) follow standard casino rules. The class also interacts with the game view component to keep players informed about the current state of play and the results of each round. This makes `BlackjackGame` the backbone, seamlessly integrating all other components and driving the game's logic forward.

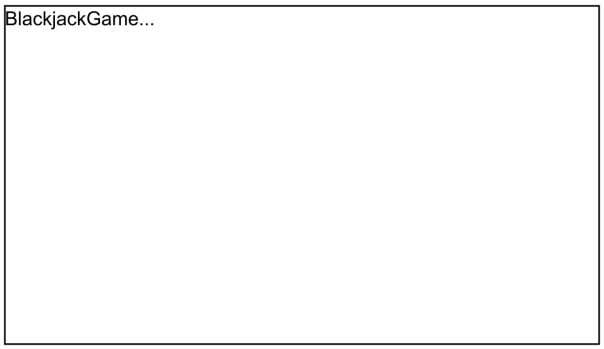


Figure 1.19: The class diagram of the BlackjackGame class

All requirements from **R5** to **R15**.

Enumerations and custom data types

The following provides an overview of the enumerations and custom data types used in this problem:

- `Suit`: The `Suit` enumeration defines the four standard suits found in a deck of cards—Hearts, Spades, Clubs, and Diamonds. It provides a type-safe way to assign and compare suits across the system.

- `AccountStatus`: The `AccountStatus` enumeration captures the possible states of a player or dealer account, such as active, closed, canceled, blocklisted, or none. This supports account management logic, restricting access or play when required.

- `Person`: The `Person` class stores personal identification and contact details for each participant (player or dealer) in the system. It holds fields such as name, address, city, state, zip code, and country. Person objects are associated with player accounts, supporting account management, record-keeping, and potential regulatory compliance.

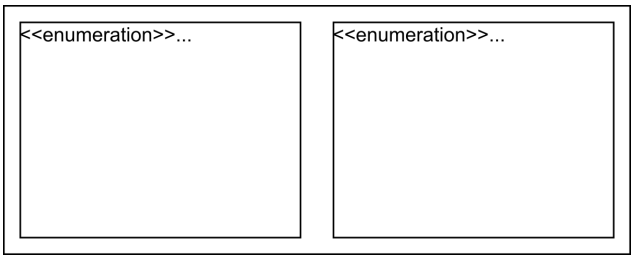


Figure 1.20: Enums in the Blackjack game

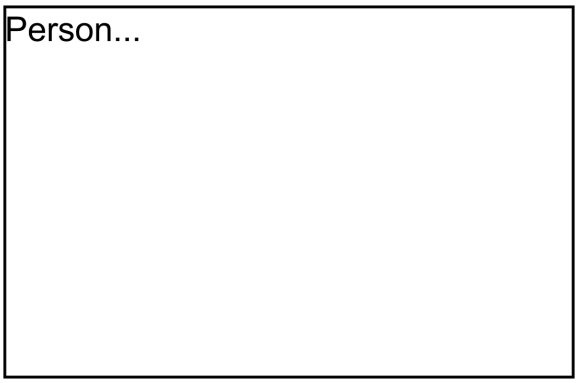


Figure 1.21: The class diagram of the Person class

4.2 Relationship between the classes

Now, we'll discuss the relationships between the classes we have defined above in our Blackjack game.

Association

The class diagram has the following association relationships:

- `BlackjackGame` has an association with `BlackjackController` and `BlackjackGameView`:
  - Calls methods to control play and to display the game state and results.
  - These are collaborating components, with the game logic relying on the controller to validate moves and the view to display outcomes.

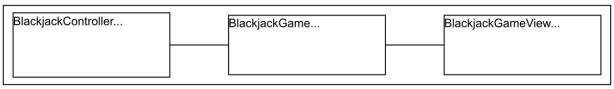


Figure 1.22: The association relationship between classes

Aggregation

The class diagram has the following aggregation relationships.

- `Shoe` is an aggregation of cards:
  - The shoe holds all cards for the current session, sourced from one or more decks. Cards exist independently of the shoe (they can be part of other components, like the deck).
- `Deck` is anaggregation ofcards:
  - A deck collects 52 cards together. The relationship is weaker than composition; cards can exist outside a deck (for example, when dealt).
- `Hand` is anaggregation ofcards:
  - A hand groups together the cards a player or dealer holds during a round.

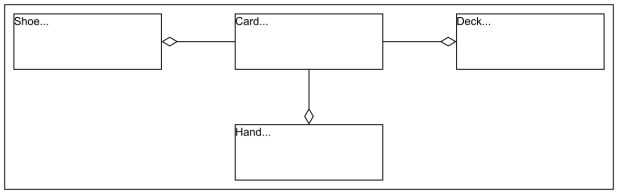


Figure 1.23: The aggregation relationship between classes

Composition

The class diagram has the following composition relationships.

- `BlackjackGame` is composed of:
  - `BlackjackPlayer`
  - `Dealer`
  - `Shoe`
  - `BlackjackController`
  - `BlackjackGameView`
- These components are created, managed, and destroyed during the game's life cycle. These objects typically cease to exist when the game ends, making this a strong whole-part relationship.
- `Player` is composed of `Hand`:
  - Each player (and dealer) owns a hand. The hand's existence is tied directly to its owner; if the player leaves, their hand is also discarded.

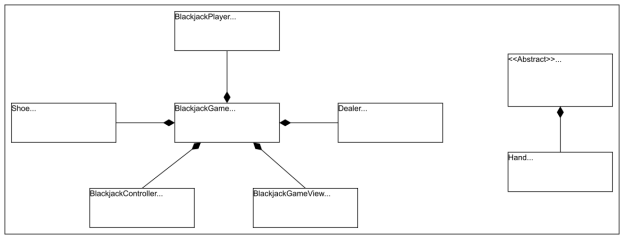


Figure 1.24: The composition relationship between classes

Inheritance

The following classes show an inheritance relationship:

- `BlackjackPlayer` and `Dealer` both *inherit* from `Player`.
  - This means both subclasses share fields such as `id`, `password`, `balance`, `status`, `person`, and `hand`, and may implement or override behaviors like `resetPassword()`.

Note: We have already discussed the inheritance relationship between classes in the component section above one by one.

4.3 Class diagram for the Blackjack game

This section outlines the multiplicity (cardinality) relationships between the main classes in our Blackjack game. We explain the allowed number of instances on each side for each relationship and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Source	Target	Multiplicity	Reason
Shoe	Deck	1–1..*	A Shoe contains one or more decks (usually 4–8 in casinos).
Deck	Card	1–52	Each Deck always contains exactly 52 cards.
Hand	Card	1–2..*	A Hand has at least 2 cards, but may have more due to hits or splits.
Player (abstract)	Hand	1–1	Each Player has exactly one Hand (may be 1–1..* if splits are supported).
BlackjackGame	Shoe	1–1	Each game session uses exactly one Shoe.
BlackjackGame	BlackjackPlayer	1–1..*	A game has one or more BlackjackPlayers.
BlackjackGame	Dealer	1–1	Each game has exactly one Dealer.
BlackjackGame	BlackjackController	1–1	Each session has one Controller handling actions.
BlackjackGame	BlackjackGameView	1–1	Each game has exactly one GameView for displaying state.

Here's the complete class diagram for our Blackjack game:

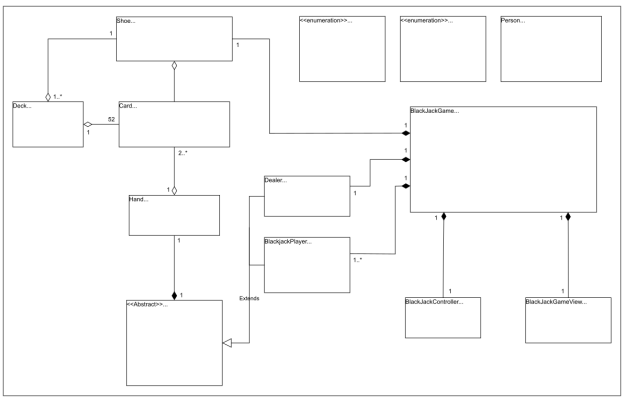


Figure 1.25: The class diagram of the Blackjack game

4.4 Design pattern

The Iterator design pattern is highly relevant to our Blackjack game. It allows sequential access to the elements in a collection—such as a deck or shoe—without exposing the underlying data structure.

> **For example:** When dealing cards from the deck or shoe, the system simply iterates through the collection of cards, handing them out to each player and the dealer. This cleanly separates the card distribution logic from the internal representation of the deck or shoe, promoting encapsulation and flexibility.

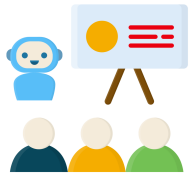
The State design pattern is also applicable and beneficial for managing the game's progression. Blackjack is a classic example of a system that transitions through a series of well-defined states. By encapsulating each state as a distinct object, the State pattern allows the game to alter its behavior when its internal state changes, without relying on complex conditionals.
Some of the core states in an online Blackjack game might include:

- **Shuffling:** The system shuffles the shoe (or decks) before starting a round.
- **Dealing:** Cards are distributed to the dealer and all players.
- **Player's Turn:** The player can choose to hit (draw a card) or stand.
- **Dealer's Turn:** The dealer reveals cards and follows house rules (e.g., must hit until reaching 17).
- **Settlement:** The game evaluates outcomes (win, lose, push), pays out winnings, and collects losses.
- **Game Over:** The round ends, and the system prepares for a new game or round.

Implementing the state pattern allows each stage to be managed as a separate state object, making the game engine more modular, maintainable, and easy to extend for additional rules or variants.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



We have completed the class diagram of the Blackjack game according to the requirements. In the next lesson, let's design the activity diagram of the Blackjack game.

5 Activity Diagram for the Online Blackjack Game

Activity diagrams are a great way to visualize the flow of messages from one activity to another in the system. We can create different activity diagrams for our Blackjack game. In this lesson, we will create an activity diagram for the steps to play Blackjack.

5.1 Blackjack gameplay

The following are the states and actions that will be involved in this activity diagram.

5.2 States

Initial state: The player places the bets, and the dealer deals the cards among themselves and the player.
Final state: There are four final states present in this activity diagram, shown below:

- The player wins 3:2 of the bet.
- The player wins with an equal bet.
- The match is tied.
- The player loses.

5.3 Actions

- The player places the bet.
- The dealer deals two cards to themselves and two to the player.
- If the total of the player's cards is 21:
  - It is a Blackjack, and the player will win and get 150% of the bet.
- If the total is greater than 21:
  - The player will lose, and the dealer will collect the bet.
- If the total of the player's card is less than 21:
  - The player will have two options: "Stand" or "Hit."
- If a player decides to "Hit":
  - The player gets another card from the dealer, and we will check the player's total again.
- If the player decides to "Stand":

- The dealer's card total will be checked.
- If it is less than 17, the dealer will get another card until the total is over 16.
- If it is over 21, the dealer immediately loses and the player gets the payout.
- If it is greater than 16 and less than 21, we will compare the totals of the player's and dealer's cards.
  - If the player's total is less than the dealer's, then the player loses and the dealer gets the payout.
  - If the player's total equals the dealer's, the player gets their bet back.
  - If the player's total exceeds the dealer's, the player wins and gets the payout.

Based on the order above, the activity diagram of a Blackjack game is given below:

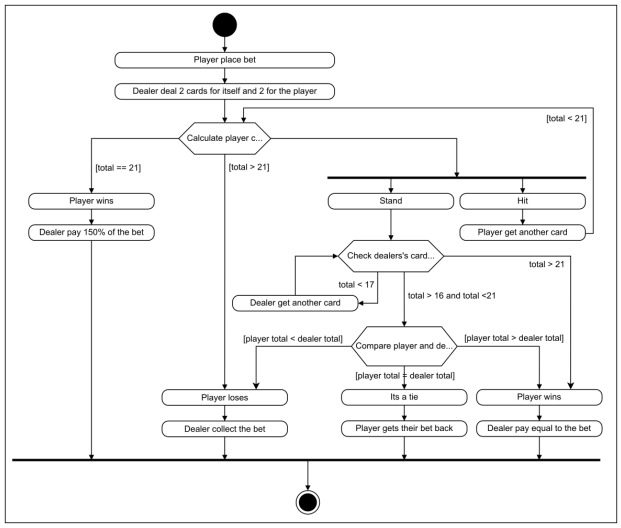


Figure 1.26: The activity diagram for the Blackjack game play

We've looked at the activity diagrams of the Blackjack game. In the next lesson, we will present the code for our designed classes in some of the most popular languages.

6 Code for the Online Blackjack Game

Using various UML diagrams, we've covered different aspects of the Blackjack game and observed the attributes attached to the problem. Let us now explore the more practical side of things, where we will work on implementing the Blackjack game using multiple languages. This is usually the last step in an object-oriented design interview process.
We have chosen the following languages to write the skeleton code of the different classes present in the Blackjack game:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click Executable Code: Online Blackjack Game.

6.1 Blackjack game classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Enumerations and custom data type

The following code defines the enumeration and custom data type used in the Blackjack game.
`Suit:` We need to create an enumeration to keep track of the card's suit, whether diamond, spade, heart, or club.
`AccountStatus:` We need to create an enumeration to keep track of the account's status, whether it is active, canceled, closed, blocked, or none.
The `Person` class is used as a custom data type. The implementation of the `Person` class can be found below:

> **Note:** JavaScript does not support enumerations, so we will use the `Object.freeze()` method as an alternative that freezes an object and prevents further modifications.

Definition of enums and custom datatypes

```
public enum Suit {
    HEART, SPADE, CLUB, DIAMOND
}

public enum AccountStatus {
    ACTIVE, CLOSED, CANCELED, BLACKLISTED, NONE
}

public class Person {
    private String name;
    private String streetAddress;
```

```
    private String city;
    private String state;
    private String country;

    // Constructor(s)
    // Getters and setters
}
```

Card

This class contains the playing cards or cards used in the Blackjack game.

The Card class

```
public class Card {
    private Suit suit;
    private int faceValue;

    public Card(Suit suit, int faceValue) { /* ... */ }
    public int getValue() { /* ... */ }
    public Suit getSuit() { /* ... */ }
    public int getFaceValue() { /* ... */ }
    @Override
    public String toString() { /* ... */ }
}
```

Deck and Shoe

`Shoe` is a device to hold multiple `Deck`, and a `Deck` has 52 cards of four suits. One suit contains nine number cards (2–10) and four face cards (King, Queen, Jack, and Ace).

The Deck and Shoe classes

```
import java.util.*;

public class Deck {
    private List<Card> cards;

    public Deck() { /* ... */ }
    public List<Card> getCards() { /* ... */ }
}

public class Shoe {
    private List<Deck> decks;

    public Shoe(int numberOfDecks) { /* ... */ }
    public void shuffle() { /* ... */ }
    public Card dealCard() { /* ... */ }
    public List<Deck> getDecks() { /* ... */ }
```

```
}
```

Hand

The `Hand` class represents a Blackjack hand used in this game and contains multiple cards.

The Hand class

```
import java.util.*;

public class Hand {
    private List<Card> cards;

    public Hand(Card card1, Card card2) { /* ... */ }
    public void addCard(Card card) { /* ... */ }
    public List<Card> getCards() { /* ... */ }
    public int getScore() { /* ... */ }
    @Override
    public String toString() { /* ... */ }
}
```

Players

The `Player` is an abstract class, and the `BlackjackPlayer` and `Dealer` classes extend the `Player` class.

- `BlackjackPlayer`: They place the first wager and update the stake with winning and losing sums. They can choose between the hit and stand options.
- `Dealer`: They are primarily responsible for dealing cards and following the Blackjack rules.

Player and its derived classes

```
public abstract class Player {
    protected String id;
    protected String password;
    protected double balance;
    protected AccountStatus status;
    protected Person person;
    protected Hand hand;

    public Player(String id, String password, double balance, Person person) { /*
        ... */ }
    public void setHand(Hand hand) { /* ... */ }
    public Hand getHand() { /* ... */ }
    public double getBalance() { /* ... */ }
```

```
    public abstract boolean resetPassword();
}

public class BlackjackPlayer extends Player {
    private double bet;

    public BlackjackPlayer(String id, String password, double balance, Person
        person) { /* ... */ }
    public void placeBet(double amount) { /* ... */ }
    public double getBet() { /* ... */ }
    public void payout(double multiplier) { /* ... */ }
    @Override
    public boolean resetPassword() { /* ... */ }
}

public class Dealer extends Player {
    public Dealer(String id, String password, double balance, Person person) { /*
        ... */ }
    @Override
    public boolean resetPassword() { /* ... */ }
}
```

Blackjack controller and game view

The `BlackjackController` class validates the actions (hit or stand) and responds accordingly. The `BlackjackGameView` class represents the game view.

The BlackjackController and BlackjackGameView classes

```
public class BlackjackController {
    public boolean validateAction(String action) { /* ... */ }
}

public class BlackjackGameView {
    public void showHands(Hand player, Hand dealer, boolean hideDealerHole) { /*
        ... */ }
    public void showResult(String result, double payout) { /* ... */ }
}
```

Blackjack game

The `BlackjackGame` class represents how we can play this game, or its basic sequence of play.

The BlackjackGame class

```
public class BlackjackGame {
    private BlackjackPlayer player;
    private Dealer dealer;
    private Shoe shoe;
```

```
    private BlackjackController controller;
    private BlackjackGameView view;

    public BlackjackGame(BlackjackPlayer player, Dealer dealer, int numDecks) { /*
        ... */ }
    public void start() { /* ... */ }
    public void playAction(String action) { /* ... */ }
    private void hit() { /* ... */ }
    private void stand() { /* ... */ }
    private void compareAndSettle() { /* ... */ }
    private void settle(double multiplier, String message) { /* ... */ }
}
```

6.2 Executable Code: Online Blackjack Game

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the Online Blackjack Game. The main driver code of the system resides in the `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the code.

What this code shows

- **System initialization:** Sets up the player with an account and balance, a dealer, and a multi-deck shoe.
- **Scenario 1 (Player gets Blackjack):** Simulates a round where the player achieves Blackjack.
- **Scenario 2 (Player busts):** Demonstrates a player drawing too many cards and losing their bet.
- **Scenario 3 (Dealer busts):** Shows the dealer exceeding 21, allowing the player to win.
- **Scenario 4 (Push):** Illustrates an outcome where players and dealers have equal scores, resulting in a tie.
- **Scenario 5 (Player wins by standing):** Player stands and beats the dealer's hand.
- **Scenario 6 (Player forfeits):** Player forfeits in the middle of the game and loses their bet.

Hint: Try customizing the code!
This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Executable Code: Online Blackjack Game

```java
import java.util.*;

public class Driver {
    public static void main(String[] args) {
        BlackjackPlayer player = new BlackjackPlayer(1000);
        Dealer dealer = new Dealer();
        BlackjackGame game = new BlackjackGame(player, dealer, 4);

        // Scenario 1: Player gets Blackjack
        System.out.println("
--- Scenario 1: Player gets Blackjack ---");
        System.out.println("
Initial balance $" + player.getBalance());
        player.placeBet(100);
        game.view.announceBet(100, player.getBalance());
        game.start(); // Initial deal
        // Normally, we'd check if player actually got blackjack, but we'll proceed
            to "stand" to demo flow.
        game.view.announcePlayerAction("stand");
        game.playAction("stand");

        // Scenario 2: Player bust
        System.out.println("
--- Scenario 2: Player bust ---");
        player.placeBet(50);
        game.view.announceBet(50, player.getBalance());
        game.start();
        game.view.announcePlayerAction("hit");
        game.playAction("hit");
        game.view.announcePlayerAction("hit");
        game.playAction("hit"); // Very likely to bust!

        // Scenario 3: Dealer busts
        System.out.println("
--- Scenario 3: Dealer busts ---");
        player.placeBet(75);
        game.view.announceBet(75, player.getBalance());
        game.start();
        game.view.announcePlayerAction("stand");
        game.playAction("stand");

        // Scenario 4: Push (tie)
        System.out.println("
--- Scenario 4: Push (tie) ---");
        player.placeBet(100);
        game.view.announceBet(100, player.getBalance());
        game.start();
        game.view.announcePlayerAction("stand");
        game.playAction("stand");

        // Scenario 5: Player stands & wins
        System.out.println("
--- Scenario 5: Player stands and wins ---");
        player.placeBet(60);
        game.view.announceBet(60, player.getBalance());
```

```
        game.start();
        game.view.announcePlayerAction("stand");
        game.playAction("stand");

        // Scenario 6: Player forfeits
        System.out.println("
--- Scenario 6: Forfeit ---");
        player.placeBet(80);
        game.view.announceBet(80, player.getBalance());
        // Forfeit simulation (player leaves):
        game.view.announceForfeit(player.getBalance());
        player.payout(-1); // Loses their bet
    }
}
```

6.3 Wrapping up

In this chapter, we've explored the complete design of the Blackjack game. We 've also examined how a basic Blackjack game can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

7 Summary: Online Blackjack Game

This lesson reinforces the core concepts of the online Blackjack game by reviewing its key requirements, system structure, and major design decisions. It reviews the key system requirements, outlines the core classes with their responsibilities and relationships, and highlights the major design principles applied. It also examines object interactions within the system and walks through the overall workflow to illustrate how the components collaborate to achieve the intended functionality.

7.1 Key requirements

The following are the main requirements for the system:

1. The system must support a shoe containing one or more standard decks, each consisting of 52 cards across four suits (Hearts, Diamonds, Clubs, Spades) and 13 ranks (Ace through King).

2. Cards must have specific point values: numbered cards equal their face value, face cards (Jack, Queen, King) are worth 10 points, and Aces are valued at 1 or 11, depending on the hand's context.

3. The system includes two user roles: Player and Dealer.

4. Players must place a bet at the beginning of each round.

5. At the start of a round, the dealer deals two cards each to the player and themselves; the player's cards are dealt face-up, while the dealer's hand shows one card face-up and one face-down.

6. Players may choose to *hit* (draw an additional card) as long as their hand total remains under 21, or *stand* to end their turn.

7. The dealer must draw additional cards (hit) until reaching a minimum hand value of 17.

8. A hand total exceeding 21 results in a bust and an immediate loss.

9. At the end of a round:

   (a) A player wins if their hand total is higher than the dealer's without exceeding 21.

   (b) A Blackjack (an Ace and a 10-point card as the initial hand) results in a 3:2 payout.

   (c) A tie (push) returns the original bet to the player or may allow a replay.

10. The dealer wins by default if a player leaves or disconnects before a round ends.

7.2 System actors

The system involves both primary and secondary actors, each with specific roles and interactions:

- **Player:** Joins games, places bets, performs actions (hit/stand/resign), and manages their account.

- **Dealer:** Manages the game flow, deals cards, enforces rules, handles payouts, and can manage player accounts.

7.3 Important classes and relationships

Below is an overview of the core classes and how they relate to one another:

Class	Description	Important Relationships
Card	Represents a single playing card with suit and value.	Used in Deck, Hand, and Shoe (Aggregation).
Deck	Represents a standard 52-card deck.	Aggregates Card; part of Shoe.
Shoe	Combines multiple Deck instances and supports shuffling.	Aggregates Deck; used by BlackjackGame.
Hand	Holds cards for a player or dealer and calculates total.	Aggregates Card; part of Player (Composition).
Player	Abstract class for common player attributes and behaviors.	Composed of Hand; associated with Person.
BlackjackPlayer	Represents the human player with bet handling.	Inherits from Player.
Dealer	Represents the dealer and enforces house rules.	Inherits from Player.
Person	Contains personal details like name and address.	Associated with Player.
BlackjackController	Validates and processes player actions.	Associated with BlackjackGame.
BlackjackGameView	Displays game state and results to players.	Associated with BlackjackGame.
BlackjackGame	Manages rounds, game state, and outcomes.	Composed of Shoe, Dealer, Players, BlackjackController, BlackjackGameView.
Suit	Enum for Hearts, Spades, Clubs, Diamonds.	Used in Card.
AccountStatus	Enum representing player/dealer account state.	Used in Player.

7.4 Design highlights

This section outlines the major design methodologies, principles, and patterns applied in the system.

7.5 Design approach

The Blackjack game was designed using a bottom-up approach, starting with low-level entities (e.g., `Card`, `Deck`, `Hand`) and progressively integrating them into higher-level components such as the `BlackjackGame`.

7.6 Design principles

Here's how each of the SOLID principles is applied:

- **Single Responsibility principle (SRP):** Each class has one well-defined responsibility. For example, a `Card` calculates its value, while `BlackjackGame` manages game flow.
- **Open/Closed principle (OCP):** Classes like `Player` can be extended via `BlackjackPlayer` and `Dealer` without modifying the base.
- **Liskov Substitution principle (LSP):** `BlackjackPlayer` and `Dealer` can be used wherever `Player` is expected.
- **Interface Segregation principle (ISP):** Though not explicitly named, responsibilities are segregated across the view, controller, and model layers.
- **Dependency Inversion principle (DIP):** High-level modules like `BlackjackGame` depend on abstractions (e.g., `Player`) rather than concrete implementations.

7.7 Design patterns

The system applies the following design patterns:

- **Iterator:** Used to sequentially deal cards from a `Deck` or `Shoe` without exposing internal structure.
- **State:** Models different game stages (e.g., shuffling, dealing, player's turn, dealer's turn, settlement, game over) as distinct, encapsulated states for clarity and extensibility.

7.8 Object interactions

This section describes how system objects collaborate to implement key gameplay functionality:

- **Game initialization:** `BlackjackGame` instantiates and wires together the `Dealer`, `BlackjackPlayers`, `Shoe`, `BlackjackController`, and `BlackjackGameView` to set up the session.
- **Bet placement:** Each `BlackjackPlayer` places a wager before the round begins, which is recorded and validated by the game engine.
- **Card dealing:** `BlackjackGame` draws cards from the `Shoe` and assigns them to the `Hand` objects of all players and the dealer at the start of the round.
- **Player turn:** `BlackjackController` validates each player's chosen action (hit or stand) and updates their `Hand` accordingly by drawing cards.
- **Dealer turn:** The `Dealer` reveals the hidden card and draws additional cards as needed until reaching a total hand value of at least 17.
- **Outcome evaluation:** `BlackjackGame` evaluates all hands, compares them against the dealer's, and determines the result for each player (win, loss, push).

- **View update:** `BlackjackGameView` clearly and timely presents the final hands, outcomes, and any payouts or losses to the players.

7.9 System workflow

This section outlines the primary end-to-end scenarios in the system:

- **Creating or logging into an account:** A player or dealer begins by registering a new account or logging into an existing one. The system verifies credentials, activates the session, and provides access to game-related features.
- **Setting up and joining a game:** The dealer creates a new Blackjack table, making it visible in the list of open games. Players browse the list and join an available game. Once the minimum number of players is reached, the game becomes ready to start.
- **Placing bets:** At the start of each round, all active players are prompted to place their bets. The system validates each wager against the player's available balance and stores the bet amounts for payout calculation.
- **Dealing initial cards:** The system draws cards from the shoe and deals two to each player and the dealer. Players receive both cards face up, while the dealer's hand shows one face up and one face down.
- **Player turn:** Each player takes their turn in sequence. They can hit (draw a card) or stand (end their turn). If a player hits, the system adds a card to their hand and checks for a bust. The turn ends when the player stands or busts.
- **Dealer turn:** After all players have acted, the dealer reveals the hidden card. Following house rules, the dealer draws cards until the hand total reaches at least 17 or a bust occurs.
- **Settlement and payout:** The system evaluates each player's outcomes once all hands are finalized. It determines wins, losses, pushes, or Blackjacks. The system calculates payouts or collects the losing bets based on the result.
- **Round conclusion and continuation:** The round results are displayed to all players through the game view. The system resets necessary states and prepares for the next round, allowing players to continue or leave.

8 Key Insights and Tips: Designing an Online Blackjack Game

You have completed the case study for designing an online Blackjack game. This lesson adds to your preparation for low-level design interview questions, especially those involving

games like Blackjack. It will highlight common mistakes to avoid, provide important tips for interviews, offer a short quiz to test what you know, and suggest other case studies that can help you understand the ideas better.

8.1 Common mistakes

To build a better and bigger online Blackjack game, try to avoid these common mistakes:

- **Lack of clear game control flow:** Not establishing a central class to manage the overall game logic, from wagers to payouts. You should implement a `BlackjackGame` class to control the game flow, accept wagers, distribute cards, and update game status.

- **Not addressing ties:** Failing to define what happens when the player and dealer have the same points. You should implement a rule allowing the player to take their bet back or replay the game in case of a tie.

- **Overlooking distinct player roles:** Not differentiating between the player and dealer roles with their specific actions and rules. You should define an abstract `Player` class with `BlackjackPlayer` and `Dealer` derived classes, each with distinct responsibilities like placing bets or dealing cards.

- **Poor handling of game states:** Not clearly defining the various phases of the game, such as shuffling, dealing, hitting, and standing. You should apply the State design pattern to manage the game's finite states, ensuring correct transitions between them.

- **Ignoring player/dealer busting rules:** Allowing hands to continue beyond 21 points without ending the game. You must implement a clear rule: if a hand exceeds 21 points, the player or dealer busts and loses the game.

8.2 Key interview tips

The following table shows some tips to help you prepare for an interview where you're asked to design an online blackjack game:

Tip	Explanation
Clarify requirements.	Explicitly confirm the rules of Blackjack with your interviewer, especially card values, hitting rules for the player and the dealer, and winning conditions. This shows attention to detail and foundational understanding.
Identify core components of the game.	Discuss logical classes like Card, Deck, Shoe, Hand, and Player early in your design. This shows a modular and object-oriented approach.
Consider extensibility for different card games.	Briefly mention how your Card, Deck, and Shoe classes could be generic enough to support other card games with minor modifications. This demonstrates forward-thinking design.
Justify your choice of design patterns.	Explain why patterns like Iterator (for drawing cards) and State (for game flow) suit this problem. This showcases understanding of clean, scalable design practices.
Be prepared to discuss scalability for multiple concurrent games.	Even if the problem focuses on one player vs. dealer, be ready to explain how your design can scale to handle many players simultaneously—e.g., by running multiple game instances or using a session manager.

8.3 Test your understanding

Take this quick quiz to check your understanding of designing an online blackjack game:

Designing an Online Blackjack Game

Question 1: Which design pattern is suitable for assigning cards to players from the deck by iterating through a list of cards?

- State design pattern
- Observer design pattern
- Strategy design pattern
- **[CORRECT]** Iterator design pattern
  Explanation: The Iterator design pattern can be applied since cards are assigned to players from the deck by iterating through the list of cards.

Question 2: The "Block member" use case involves modifying a member's status. What relationship exists between "Block member" and "Modify member" use cases?

- Include

- **[CORRECT]** Extend
  Explanation: The "Block member" use case has an extend relationship with the "Modify member" use case, as blocking membership specifically modifies the member status.
- Composition
- Association

Question 3: What type of UML relationship exists between the `Player` and `Hand` classes?

- Association
- Aggregation
- **[CORRECT]** Composition
  Explanation: The Player class is composed of Hand, indicating that a Hand is an integral part of a Player and typically doesn't exist independently.
- Inheritance

8.4 Related case studies

Explore these related case studies to deepen your understanding of the design principles applied in the online Blackjack game:

- **Designing an online stock brokerage system:** This involves managing real-time transactions and account balances, dealing with high concurrency, similar to managing bets and payouts in a game.
- **Designing an online chess game:** This requires managing game states, player moves, and turns, and validating game rules, which parallels state management and rule enforcement in Blackjack.
- **Designing an online gaming platform (General):** This broader problem encompasses managing multiple concurrent games, player accounts, leaderboards, and real-time interactions, which are scalable extensions of the Blackjack game.
- **Designing a digital wallet/payment system:** This involves handling financial transactions, managing balances, and ensuring secure and accurate money transfers, directly relevant to betting and payouts.
- **Designing a fantasy sports league:** Requires managing player data, team compositions, scoring rules, and real-time updates based on external events, similar to tracking game state and player hands.